Where TLS Fingerprinting Tools Stop Working: Seven Tests of TShark and FoxIO ja4, and a Stateful QUIC Initial Extractor

Suhani
Roll No. 24110358

Miloni Thakkar
Roll No. 24110204

Yash Goyal
Roll No. 24110399

Laxmidhar Panda
Roll No. 24110185

I. Introduction

Most web traffic is encrypted, but the first message of a TLS connection, the ClientHello, is sent in plaintext. Each TLS library fills it in a slightly different way: which cipher suites it offers, which extensions, in what order. A hash of these fields therefore works as a fingerprint of the client software. JA3 (Salesforce) and the JA4+ family (FoxIO) are the fingerprints in common use, and firewalls, intrusion detection systems and CDNs use them to recognise applications without decrypting anything. JA4+ also extends the idea below and above TLS: JA4T fingerprints the TCP SYN, JA4X the server certificate and JA4H the HTTP request.

In Phase 1 we wrote our own JA3/JA4 tool. For Phase 2 we were asked not to build another one, but to take the tools that are used in practice, Wireshark/TShark (written in C) and FoxIO's ja4 (written in Rust), and find where they stop giving a correct or useful answer. Protocols are changing under these fingerprints: QUIC moves the ClientHello into encrypted UDP packets, TLS 1.3 encrypts the certificate, post-quantum key shares make the ClientHello larger, browsers insert random GREASE values, and ECH hides the server name.

We did this in two parts. In Track 1 we captured real traffic in seven situations, changing one thing at a time, and ran TShark and FoxIO on each capture. In Track 2 we wrote our own QUIC Initial extractor in C++ and compared its output with TShark byte for byte.

Our main findings are: (1) both tools read QUIC ClientHellos correctly, so our starting hypothesis for QUIC was wrong; (2) without TLS keys, JA4X and JA4H give nothing on TLS 1.3; (3) TCP-level values describe the TCP stack nearest to the capture point, which in our labs was Docker or Windows rather than the client; (4) a post-quantum key share changes JA4 through side effects; (5) the same 50 Chromium ClientHellos give 15 different JA4 values or 1, depending on which tool version computes them; (6) ECH hides the name in TLS but plain DNS still shows it. In Track 2 we found that an extractor which derives keys from each packet's own connection ID fails on the client packets sent after the server switches connection IDs, and we added a stateful mode that handles this, with a measured weakness when packets are reordered.

II. Problem Statement

The question is: given real captures of client traffic, when do the fingerprints produced by TShark and FoxIO ja4 stop being correct, available, or stable?

The input is a pcap of real traffic from a known client (Chromium, curl or Python) to a known server (Cloudflare), captured under one controlled condition. The output is the set of fingerprints the tools produce for it, together with the values they are computed from, and a comparison against a control capture in which only that one condition is different.

We take the point of view of a passive observer on the network, such as a firewall. It sees the packets but has no TLS session keys, except in one test where we give the keys on purpose to see what changes. We do not modify the tools; we measure them as released, and record their versions.

In scope: QUIC, TLS keys, capture point and segmentation offload, post-quantum key shares, repeated connections from one client, IPv6, and ECH with encrypted DNS; and, for QUIC, recovering the ClientHello from Initial packets ourselves. Out of scope: TLS 1.2, QUIC Retry, connection migration and QUIC v2, and building a new fingerprint.

The gap is that the fingerprint specifications describe how to compute a value from a ClientHello, but not how the value behaves when the transport, the capture point, the key exchange or the tool version changes. For QUIC there is an additional practical gap: the Initial keys must come from the client's first connection ID even after the server switches IDs (RFC 9001 Section 5.2, RFC 9000 Section 7.2), and an extractor that derives keys per packet misses those packets.

III. Project Objectives

1. For JA3, JA3S, JA4, JA4S, JA4T, JA4X and JA4H, list the fields each one reads, how they are combined, which RFC defines each field, and what a passive observer can see without keys.
2. Run seven tests (L1 to L7), each changing exactly one variable against a control, on real captures, and

report one main number per test from TShark and FoxIO output.

3. Build a C++ extractor that recovers every complete QUIC v1 ClientHello in our two QUIC captures (5 per capture) with the same bytes as TShark.
4. Decrypt every client Initial packet in those captures, including the ones sent after the connection ID switch (20 for curl, 16 for Chromium).
5. Measure how the extractor behaves on damaged copies of the captures (reordered, dropped and truncated packets), compared with TShark on the same damaged file.
6. Make every number reproducible from the repository: committed pcaps with SHA-256 hashes, scripts, tests and CI.

V. Architecture and Design

The project has two pipelines that share the same committed captures.

Track 1 (reference tools). A Docker image (cn-lab, Debian trixie) pins the tools: TShark 4.4.19, FoxIO ja4 at tag v0.18.6, and FoxIO's main branch at commit c6469a7 built separately as ja4-main. A capture script runs one recipe at a time (for example, curl with only the X25519 group, five times) while tcpdump records on the container interface. An analysis script then runs TShark and FoxIO on every pcap and writes one summary row per capture: number of ClientHellos found by each tool, distinct JA3 and JA4 values, ClientHello size, number of TCP segments or CRYPTO frames, chosen group, and the JA4X/JA4H counts with and without keys. Every pcap is registered in a manifest with its SHA-256 and a metadata file describing the condition.

Track 2 (our QUIC extractor). The extractor is a pipeline of small C++ components:

```
pcap file -> capture_reader (Ethernet, IPv4/IPv6, UDP) -> packet_parser (QUIC long header, DCID, SCID) -> initial_context (stateful mode only) -> initial_crypto (Initial keys, header protection, AES-128-GCM) -> frame_parser and crypto_reassembly (CRYPTO frames by offset) -> client_hello (checked ClientHello bytes).
```

We made three design decisions. First, the extractor outputs ClientHello bytes and does not compute JA4. FoxIO already computes JA4, and a second implementation of our own would only add another thing to get wrong; comparing raw bytes with TShark is also a stricter check than comparing hashes. Second, the only library dependency is OpenSSL 3, used for HKDF and AES-GCM, so that we do not write any cryptography ourselves. Third, the stateful mode keeps very little state: for each UDP 4-tuple, the first client destination connection ID, the client's source ID and the server source IDs learned from server Initials. It does not track the whole connection.

The trade-off of the stateful design is order sensitivity. It assumes the first client Initial seen on a 4-tuple is the real first one. This is true for captured traffic, but if packets are reordered a later packet becomes the key source and the real first packet is rejected. The stateless mode does not have this problem but cannot decrypt the post-switch packets. We kept both modes so they can be compared.

VI. Implementation

The extractor is written in C++20 and built with CMake; OpenSSL 3 provides the EVP interfaces for HKDF and AES-128-GCM. The sources are in code/quic/src: capture_reader.cpp reads classic pcap files, packet_parser.cpp parses QUIC long headers, initial_crypto.cpp derives the Initial secrets (RFC 9001 Section 5.2), removes header protection (Section 5.4) and decrypts, frame_parser.cpp and crypto_reassembly.cpp collect CRYPTO frames and place them by offset (identical overlaps accepted, conflicting overlaps rejected, with a size limit), and client_hello.cpp checks the ClientHello structure. Key derivation is tested against the values in RFC 9001 Appendix A.

The stateful association (initial_context.cpp) works as follows:

for each client Initial on flow (client ip:port, server ip:port): if it is the first packet on the flow, remember its DCID as the key source; if its DCID is the key source or a server SCID learned on this flow, decrypt with keys from the key source; otherwise do not associate it. For each server Initial whose DCID is a known client SCID, remember its SCID as an alias for the flow.

The Track 1 and verification tooling is in Python (code/scripts): capture.py and analyze.py for Track 1, quic_initial_inventory.py for a header-only list of connection IDs, quic_impairment.py for the damaged copies, l3_proxy_sanity.py for SYN and SYN-ACK values, and verify_submission_batch.sh, which runs all Track 2 checks into a dated folder. The C++ code has one CTest program and the Python code has 52 pytest tests; GitHub Actions runs both, plus the manifest hash check, on every push.

Challenges. The first version of the extractor reported decryption failures for 10 of 20 curl and 6 of 16 Chromium client Initials, while TShark decrypted them (we checked this on the curl capture with TShark 4.6.8). A header-only inventory of the captures showed that in each of the five flows per capture, the client's later Initials carry the server's 20-byte connection ID instead of the client's original 8-byte one. The fix was the stateful mode above. A second problem was in our comparison script: TShark's jsonraw output can repeat a key, and a plain JSON parser keeps only the last value, so repeated handshake values were lost; we changed the parser to keep them. Third, the imported Track 1 summary reported the two tools as agreeing on the IPv6 capture, which turned out to contain no packets at all; we mark it as not comparable.

Repository:
https://github.com/milonithakkar/CS331-Phase2

VII. Testbed Setup

Track 1 lab. Docker Desktop on a MacBook (aarch64, Linux kernel 7.0.12-linuxkit inside the VM), on the IITGN campus network. Inside the cn-lab container: TShark 4.4.19, FoxIO ja4 tag v0.18.6 (the binary reports 0.18.5), ja4-main at commit c6469a7 (reports 0.18.6), tcpdump 4.99.5, Chromium 156.0.8078.79, curl 8.14.1 with OpenSSL 3.5.7 (HTTP/3

through OpenSSL's QUIC), Python 3.13.5. Captures were taken with tcpdump on the container's eth0. For the offload test the link MTU was set to 1500 and TSO/GSO were switched with ethtool.

Track 2 and the L3 paired capture. A Windows 11 laptop (Intel Core Ultra 7 256V) with Ubuntu on WSL2 (kernel 6.18.33.2, 7.7 GB of memory visible to WSL), TShark 4.6.4, GCC 15.2.0, OpenSSL 3.5.5, Python 3.14.4, and Docker Desktop. For L3 the same requests were captured at the same time inside a Docker container (eth0), on WSL's eth0, on the Windows vEthernet (WSL) adapter and on the Wi-Fi adapter.

Re-check. On 6 November the Track 1 numbers that the summary does not store were recomputed on the committed pcaps with TShark 4.6.8 on macOS and FoxIO tag v1.0.1 (released 31 August 2026).

Workloads. All servers are Cloudflare: cloudflare-quic.com and www.cloudflare.com/cdn-cgi/trace for QUIC and TCP, and crypto.cloudflare.com/cdn-cgi/trace for ECH, which reports whether the SNI arrived encrypted. Each test repeats the request 1 to 10 times (Chromium page loads in L5 produced 50 ClientHellos). In total there are 16 pcaps: 15 Track 1 captures and one earlier QUIC baseline capture. The damaged copies use a fixed random seed (331).

VIII. Evaluation

Table I gives one line per test. The commands and full outputs are in the repository under docs/tests and results.

TABLE I. Seven tests with TShark and FoxIO ja4

	Change	Result	Status
L1	QUIC instead of TCP	5/5 ClientHellos in both tools for curl and Chromium (21 CRYPTO frames for Chromium)	Tools work
L2	No TLS keys	JA4X + JA4H: 0 without keys, 5 with keys	Confirmed
L3	Offload, capture point	1 frame of 1586 B vs 1514 + 138 B; SYN-ACK and SYN values from the nearest TCP stack	Confirmed in our labs
L4	PQ key share	1511 B vs 508 B; JA4 12 vs 14 extensions	Confirmed
L5	50 Chromium hellos	50 JA3; JA4: 15 (TShark, FoxIO v0.18.6) or 1 (FoxIO v1.0.1)	Confirmed
L6	IPv6	Capture has 0 packets	Not measured
L7	ECH	SNI cloudflare-ech.com, DNS still shows crypto.cloudflare.com	Confirmed; DoH not measured

L1, QUIC. Both TShark and FoxIO found all 5 ClientHellos for curl and for Chromium, including Chromium's ClientHello split over 21 CRYPTO frames, and gave the same JA4. Our hypothesis that the tools would fail here was wrong. QUIC Initial encryption does not hide the ClientHello from a passive observer, because the keys come from the connection ID in the packet.

L2, keys. On one TLS 1.3 connection, FoxIO produced 0 JA4X and JA4H values without the key log and 5 with it (4 certificates and 1 HTTP request). On TLS 1.3 these two fingerprints are only available when the keys are.

L3, capture point. With offload on, the ClientHello appears as one 1586-byte frame, larger than the 1500-byte MTU; with offload off, as 1514 + 138 bytes. JA4 is the same. In the Mac lab, the SYN-ACK seen inside the container had MSS 65495; in the Windows run, Cloudflare's SYN-ACK on the Wi-Fi adapter had MSS 1400 and window scale 13 while the container saw MSS 1460 and window scale 7, so the SYN-ACK inside the container is produced by Docker's network stack, not by the server. Table IV shows the client side.

L4, post-quantum key share. Offering only X25519MLKEM768, the ClientHello is 1511 bytes; offering only X25519, 508 bytes. The servers chose group 4588 and 29 respectively. JA4 has no field for groups, but it still changed (12 against 14 extensions): with only the PQ group OpenSSL leaves out ec_point_formats, and the small classic hello gets the padding extension.

L5, one browser. 50 ClientHellos from Chromium gave 50 different JA3 values, because Chromium shuffles its extensions. For JA4 the count depends on the tool: 15 with TShark 4.4.19, TShark 4.6.8 and FoxIO v0.18.6, and 1 with FoxIO v1.0.1 and FoxIO main. The 15 correspond to 15 different GREASE values that Chromium places first in its signature algorithms. FoxIO's fix (commit d66336e) ignores GREASE there; TShark 4.6.8 does not. curl and Python, both on OpenSSL 3.5.7, each gave a single JA3 and JA4, and share JA4 part b but not parts a and c.

L6, IPv6. Neither lab had an IPv6 route, and the capture contains no packets. We report no result.

L7, ECH. With ECH on, the SNI on the wire is cloudflare-ech.com and the page reports the SNI as encrypted, but the plain DNS query for crypto.cloudflare.com is still in the capture. ECH adds one extension, so JA4 changes from 16 to 17 extensions. The run with DNS over HTTPS did not load the page on the campus network, so that case is not measured.

Track 2. Table II shows the extractor on the captures in their original order. The stateful mode decrypts every client Initial, where the stateless mode decrypts 10 of 20 and 10 of 16. In both modes all 5 ClientHellos per capture are identical to

TShark's. In the curl capture we checked that these extra packets carry only ACK and PADDING frames, and in both captures the 5 ClientHellos are complete without them, so the ClientHello, and therefore JA4, does not change between modes; TShark already handles this case, and the stateful mode makes our extractor match it.

TABLE II. C++ EXTRACTOR ON THE CAPTURES IN ORIGINAL ORDER

	curl	Chromium
Client Initial packets	20	16
Decrypted, stateless	10	10
Decrypted, stateful	20	16
ClientHellos equal to TShark	5 / 5	5 / 5

Table III shows the damaged copies. The stateless mode matched TShark on all 54 ClientHellos. The stateful mode matched 42: it fails on reversed order (0 of 5) and shuffled order (4 of 5), because a later packet seen first becomes the key source. The numbers were the same for curl and Chromium. Two caveats limit what this table shows. When every third CRYPTO packet is dropped, TShark itself finds only 2 ClientHellos, so 2 of 2 means agreement with TShark, not recovery. And in the single-drop and truncation cases TShark still finds all 5 ClientHellos, so the damaged packet did not affect any ClientHello.

TABLE III. CLIENTHELLOS EQUAL TO TSHARK ON DAMAGED COPIES (PER CAPTURE)

Damage	Stateless	Stateful
None	5 / 5	5 / 5
Reversed order	5 / 5	0 / 5
Shuffled order	5 / 5	4 / 5
One CRYPTO packet dropped	5 / 5	5 / 5
Every third CRYPTO packet dropped	2 / 2	2 / 2
One packet cut by 64 bytes	5 / 5	5 / 5

Table IV is the client SYN of the same request, seen at two points in the Windows run. Every value that differs is a Windows default, and the sequence numbers also differ, so the connection that leaves the laptop is opened by Windows rather than passed through from the container. A JA4T taken on the Wi-Fi would therefore describe Windows, not the Linux client. Traffic from a Linux VM on the vEthernet adapter, in the same capture, kept its Linux values with only the TTL reduced by one, which is what a normal NAT does.

TABLE IV. CLIENT SYN OF THE SAME REQUEST AT TWO CAPTURE POINTS (WINDOWS RUN)

Client SYN	Inside container	On Wi-Fi adapter
TTL	64	128
Window	64240	65535
Window scale	10	8
MSS	1460	1460
TCP timestamps	yes	no

IX. SUMMARY

We tested where TShark and FoxIO ja4, the fingerprinting tools used in practice, stop giving correct or useful answers, using seven one-variable tests on real captures and a C++ QUIC Initial extractor checked against TShark. The tools handle QUIC correctly. The problems we found are elsewhere: JA4X and JA4H need keys that a passive observer never has on TLS 1.3; TCP fingerprints describe the nearest TCP stack, which can be Docker or the host operating system; a post-quantum key share and ECH both change JA4 through extra or missing extensions; and the JA4 of one browser depends on the tool version, 15 values or 1 for the same 50 ClientHellos, so a JA4 list has to record which tool produced it. ECH hides the server name only if DNS is encrypted too. Our extractor recovers the same ClientHello bytes as TShark, and its stateful mode decrypts the post-switch Initial packets, at the cost of failing when packets are reordered.

Limitations. Track 1 was captured inside Docker on one Mac on one network, with no IPv6 route and DNS over HTTPS blocked. We used one server operator (Cloudflare) and three clients (Chromium, curl, Python), and did not test TLS 1.2. The L3 paired run has three requests and no capture on a separate physical tap. The with-keys result of L2 cannot be regenerated from the repository because the key log is deliberately not committed. TShark is our reference for the extractor, so agreement with it is not independent ground truth.

Future work. In the stateful mode, fall back to the packet's own connection ID when the stored one fails, so that reordering does not lose ClientHellos. Add QUIC Retry handling (RFC 9001 Section 5.2 and the Retry integrity tag). Repeat L3 and L6 on a native Linux host with IPv6, and repeat L7 on a network that allows DNS over HTTPS.

IX. REPRODUCIBILITY

All code, captures and results are at https://github.com/milonithakkar/CS331-Phase2 (branch main). The C++ build needs CMake, a C++20 compiler and the OpenSSL 3 headers (libssl-dev); the Track 1 commands need Docker; the Track 2 checks need TShark.

```
git clone
https://github.com/milonithakkar/CS331-Phase2.git
&& cd CS331-Phase2
```

```
make test (builds the C++ code and runs CTest and
the 52 pytest tests)
make check (checks all 16 pcaps against their
SHA-256 in captures/manifest.json)
make lab-image && make lab-eval (rebuilds the
Docker lab and reruns TShark and FoxIO on every
Track 1 pcap)
./build/code/quic/src/quic-fingerprint --pcap
captures/pcaps/quic_curl.pcap
[--stateful-initial-context] (runs the extractor
on one capture)
code/scripts/verify_submission_batch.sh (reruns
all Track 2 checks, including the damaged copies,
into a new folder under results/track2)
```

The numbers in Section VIII come from results/summary.md, results/track1/recheck-20261106.md, results/l3_paired_batch and results/track2/verification-batch-20261105-105416.

References

[1] M. Thomson and S. Turner, “Using TLS to Secure QUIC,” RFC 9001, May 2021.

[2] J. Iyengar and M. Thomson, “QUIC: A UDP-Based Multiplexed and Secure Transport,” RFC 9000, May 2021.

[3] E. Rescorla, “The Transport Layer Security (TLS) Protocol Version 1.3,” RFC 8446, Aug. 2018.

[4] D. Benjamin, “Applying Generate Random Extensions And Sustain Extensibility (GREASE) to TLS Extensibility,” RFC 8701, Jan. 2020.

[5] E. Rescorla, K. Oku, N. Sullivan, and C. A. Wood, “TLS Encrypted Client Hello,” RFC 9849, Mar. 2026.

[6] W. Eddy, Ed., “Transmission Control Protocol (TCP),” RFC 9293, Aug. 2022.

[7] P. Hoffman and P. McManus, “DNS Queries over HTTPS (DoH),” RFC 8484, Oct. 2018.

[8] FoxIO, “JA4+ Network Fingerprinting,” tags v0.18.6 and v1.0.1. https://github.com/FoxIO-LLC/ja4

[9] Salesforce, “JA3 - A method for profiling SSL/TLS Clients.” https://github.com/salesforce/ja3

[10] Wireshark Foundation, “Wireshark / TShark,” versions 4.4.19, 4.6.4 and 4.6.8. https://www.wireshark.org

Appendix A: Team Work Distribution and Member Rating

A.1 Work Distribution

- *Ratings are agreed by the whole team and must be UNIQUE no two members may receive the same rating (no ties).*
- *Scale: 1–10 (10 = highest contribution). With 3 members, use three different values (e.g., 10, 9, 8).*

#	Member Name	Roll No.	Tasks Done	Rating (1–10, unique)
1	Suhani	24110358	Co-designed the architecture and the Phase 2 plan; wrote the fingerprint landscape and field cards (docs/landscape.md, docs/basics); checked the repository's claims against the committed data and fixed inconsistencies; rewrote the documentation; wrote the report.	10
2	Miloni Thakkar	24110204	Wrote the C++ QUIC extractor and its stateful mode; built the WSL test setup and ran the verification batches; ran the Windows paired capture for L3; integrated the repository.	9
3	Yash Goyal	24110399	Built the Docker lab; made the Track 1 captures; wrote the capture and analysis scripts and the test write-ups; re-checked the Track 1 numbers on 6 November.	8
4	Laxmidhar Panda	24110185	Analysed the TCP values for L3 (MSS, TTL, window scale); prepared the presentation.	7

Appendix B: AI Usage Details

B.1 Prompts

The four AI tools used for the most significant parts of the work are listed below. Prompts and responses are kept as .md files in the ai folder of the repository.

#	AI Tool	Purpose
1	Google Antigravity (Miloni)	Pair programming on the C++ extractor and the stateful mode, debugging the TShark JSON parser, WSL batch scripts
2	Arena agent (Miloni)	Writing C++ and Python code from the implementation plan, repository review and consistency checks
3	Claude Code, Claude Opus (Yash)	Docker lab, capture and analysis scripts, test write-ups, the 6 November re-check, consistency review of report and slides
4	Claude, Claude Opus 5.5 in Claude Cowork (Suhani)	Reviewing the repository for inconsistencies, rewriting the README and documentation, drafting this report from the repository data

AI prompts and responses: https://github.com/milonithakkar/CS331-Phase2/tree/main/ai